FIELD RUNBOOK · READ BEFORE TOUCHING THE BOARD

Jetson Bring-Up: Check, then Flash

Tanna TechBiz **Eagle 101** + NVIDIA Jetson Nano production module
Unit serial **1010425035310** · EchoPath deployment

PART A · CHECK	PART B · FLASH	FLASH NEEDED?	RISK OF BRICKING
30 min, free	Half a day	Unknown — test first	Low, recoverable

Read this first

Flashing may not be necessary. Some vendors ship the module pre-programmed. Part A is a 30-minute test that tells you which path you are on — and it may save you sourcing a Linux machine entirely. Do not start Part B until Part A says you must.

Reference · Your hardware and versions

THE BOARD

Item	Value	Why it matters here
Module	Jetson Nano **production** (P3448-0020)	Has eMMC ⇒ **no SD-card image exists**. Must be flashed over USB recovery. (The devkit module, P3448-0000, is the one you can just write an image for.)
SoC / GPU	Tegra X1, Maxwell, **SM 5.3**	128 CUDA cores. **No INT8** — no DP4A instruction.
RAM	4 GB LPDDR4, shared CPU/GPU	Needs headless boot + 8 GB swap before any TensorRT engine build.
Storage	16 GB eMMC (soldered) + microSD slot	Full JetPack ≈14 GB. Bulk must live on the card.
Camera	2× MIPI CSI-2, 2-lane, 15-pin	IMX219. The camera test in A5 is what proves the vendor BSP is installed.
USB	4× USB 3.0 A; 1× **Type-B (data only)**	Type-B is the recovery/flash port.
Audio	**None** — no analog codec	USB dongle mandatory. Audio exists only inside HDMI/DP otherwise.
Power	5 V / 4 A barrel	An under-powered supply causes random reboots that look like software faults.

SOFTWARE VERSIONS — FIXED BY THE PLATFORM, NOT CHOSEN

Component	Version	Note
JetPack	**4.6.x**	Terminal release for this module. No JetPack 5 or 6.
L4T	R32.7.x	Confirm with `cat /etc/nv_tegra_release`
Ubuntu	18.04	On the board
Python	**3.6**	Hard floor. torch 2.x and ultralytics require ≥3.8 and cannot be installed.
CUDA	10.2	
cuDNN	8.2	

Component	Version	Note
TensorRT	8.2	Engines are not portable across TRT version or GPU — build them on the board.
OpenCV	4.1.1 (JetPack)	Built with GStreamer. **Never** `pip install opencv-python` — it shadows this and the CSI camera silently stops working.
NumPy	1.19.5	Last version supporting Python 3.6. Inherited from the system.

WHAT YOU INSTALL YOURSELF

Package	Version	Source
`pycuda`	**2021.1**	pip. Last release supporting Python 3.6. Needs `PATH=/usr/local/cuda/bin:$PATH` and `CUDA_ROOT=/usr/local/cuda` to build.
`pyttsx3`	2.90	pip. Fallback only — the alert vocabulary is pre-rendered on the workstation.
`espeak`, `libespeak1`	—	apt. pyttsx3's Linux backend.
`alsa-utils`	—	apt. Provides `aplay` for the alert WAVs.

HARDWARE YOU HAVE VS STILL NEED

Item	Status	Note
128 GB SanDisk Ultra microSDXC A1	have	Adequate — A2's advantage needs host features the Nano lacks. Verify authenticity with `f3` / `h2testw` first.
HP 64 GB USB 3.2 pendrive	have	File transfer and card backups. **Not** suitable as rootfs — poor random write, wears out.
5 V / 4 A barrel supply	check	Not a phone charger.
USB audio dongle	**need**	Mandatory — no analog codec on the board.
IMX219 CSI camera + ribbon	**need**	Chest-mount length.
USB-A to Micro-B cable	**need**	Must be *data* capable. Charge-only cables are the commonest Part B failure.
x86_64 host, Ubuntu 18.04/20.04	**need if flashing**	Bare metal or dual boot. A university lab machine for one afternoon is enough.

Part A · Find out whether you need to flash

30 minutes · costs nothing · may make Part B unnecessary

A1 Gather

- ☐ Eagle 101 with the Nano module seated
- ☐ **5 V / 4 A barrel supply** — not a phone charger
- ☐ HDMI cable and monitor
- ☐ USB keyboard (mouse optional)
- ☐ microSD — leave it *out* for now

A2 Confirm the module is seated

If you assembled it: the module inserts into the SO-DIMM slot at roughly 20°, then presses flat until both side clips click. Confirm the **fan is plugged into its header**.

A partially seated module produces no display at all — identical in appearance to a blank module. Rule this out before concluding anything.

A3 First power-on

Connect HDMI **first**, then keyboard, then power. Wait up to 2 minutes — first boot is slow.

What you see	Meaning
Nothing at all, fan still	Re-check the power supply and module seating, then treat as blank
Fan spins, no display	Likely blank — bootloader area empty
NVIDIA logo → text → login/desktop	**Pre-flashed** → continue to A4

A4 If it booted — identify what is installed

```
cat /etc/nv_tegra_release      # expect R32, revision 7.x
lsb_release -a                 # expect Ubuntu 18.04
python3 --version              # expect 3.6.x
df -h /                        # HOW MUCH eMMC IS FREE?  write this down
free -h
dpkg -l | grep -i tensorrt     # is TensorRT already installed?
```

The `df -h /` figure decides a lot. Under ~3 GB free means the full JetPack is installed and you will fight for space continuously — reflashing with rootfs on the SD may be less work than working around it.

A5 The camera test — the decisive one

With the board **powered off**, connect the IMX219 to CSI port 0. Check ribbon orientation against the vendor's assembly guide — it is easy to reverse.

```
ls /dev/video*

gst-launch-1.0 nvarguscamerasrc num-buffers=100 ! \
  'video/x-raw(memory:NVMM),width=1280,height=720,framerate=30/1' ! \
  nvvidconv ! fakesink -v
```

Result	Meaning
`/dev/video0` exists, pipeline runs	**Tanna's BSP is installed. No flashing needed.**
No device, or `Failed to create CaptureSession`	Stock NVIDIA L4T — wrong device tree for this carrier. **Flash with the vendor BSP.**

Also check audio while you are here (dongle plugged in): `aplay -l`

A6 Decide

A3 boot	A5 camera	Action
No	—	**Flash** → Part B
Yes	Works	**Skip flashing.** Do A7 instead
Yes	Fails	**Flash** with the vendor BSP → Part B

A7 If skipping the flash — move the bulk onto the SD

```
lsblk                                     # find the card, expect mmcblk1
sudo mkfs.ext4 /dev/mmcblk1p1             # ONLY if the card is empty
sudo mkdir -p /mnt/sd && sudo mount /dev/mmcblk1p1 /mnt/sd
echo '/dev/mmcblk1p1  /mnt/sd  ext4  defaults,noatime  0  2' | sudo tee -a /etc/fstab

# swap on the card, NEVER on the 16 GB eMMC
sudo fallocate -l 8G /mnt/sd/swapfile && sudo chmod 600 /mnt/sd/swapfile
sudo mkswap /mnt/sd/swapfile && sudo swapon /mnt/sd/swapfile
echo '/mnt/sd/swapfile  none  swap  sw  0  0' | sudo tee -a /etc/fstab

sudo systemctl set-default multi-user.target && sudo reboot   # frees ~1 GB of 4 GB
```

Then keep `onnx/`, `engines/` and the project under `/mnt/sd/`. `noatime` matters: without it every file *read* also writes to the card.

Part B · Flashing

Only if Part A said so · half a day including JetPack install

B1 Get the BSP from Tanna — do this first

Scan the **"How to Flash JetPack (BSP)"** QR code on the retail box. If the docs are thin, email `sales@tannatechbiz.com` quoting serial **1010425035310** and ask for:

1. The BSP archive for JetPack 4.6.x
2. **The exact `flash.sh` board-configuration name for the Eagle 101**
3. Whether they support rootfs on microSD, and the invocation
4. Recovery-mode instructions — which jumper or pads, and which pins
5. *Do you offer a pre-flashing service?* — if yes, Part B disappears

> **Do not proceed without item 2**
> The board config selects the device tree: pin mux, CSI lane routing, fan control, power sequencing. A generic `jetson-nano-emmc` copied from a forum post may flash "successfully" and leave you with no camera and no fan, or fail to boot. **This is the one step where guessing causes real damage.**

B2 Prepare the host machine

x86_64, Ubuntu 18.04 or 20.04, bare metal or dual boot.

Not a virtual machine. During flashing the board disconnects and reappears on the USB bus several times as it switches between recovery and bootloader states. VM USB passthrough commonly drops it mid-transfer. WSL2 has no native USB device access at all.

Also needed: ~40 GB free disk, and a **data-capable USB-A to Micro-B cable**. Many cables are charge-only — if `lsusb` shows nothing at B5, suspect the cable before anything else.

A university lab machine for one afternoon is sufficient. This is a one-time need.

B3 Host prerequisites

```
sudo apt update
sudo apt install -y qemu-user-static abootimg nfs-kernel-server \
                    libxml2-utils binutils python3 device-tree-compiler \
                    lbzip2 cpio udev dosfstools
```

B4 Unpack the BSP

Follow Tanna's layout exactly. The conventional L4T structure is:

```
mkdir -p ~/nvidia && cd ~/nvidia
tar xf <Tanna_BSP>.tbz2                    # creates Linux_for_Tegra/
cd Linux_for_Tegra/rootfs
sudo tar xpf ../../<Tegra_Linux_Sample-Root-Filesystem>.tbz2
cd ..
sudo ./apply_binaries.sh                    # merges NVIDIA + vendor overlays
```

`apply_binaries.sh` must finish without errors. If it fails, stop — flashing a broken rootfs wastes an hour.

Optional, skips the first-boot setup wizard:

```
sudo ./tools/l4t_create_default_user.sh -u echopath -p <password> -a --accept-license
```

B5 Put the board into recovery mode

1. Power off; unplug the barrel jack
2. Short `FC_REC` **(force recovery) to** `GND` — usually a jumper or two header pads. **Use Tanna's documentation for the exact location**; it differs between carriers
3. Keep it shorted and apply power
4. Wait 5 seconds, then remove the jumper
5. Connect the USB Micro-B cable to the host

```
lsusb | grep -i nvidia
# expect:  Bus 001 Device 0xx: ID 0955:7f21 NVIDIA Corp. APX
```

> **No APX device means you are not in recovery**
> Check, in order: the jumper position, whether the USB cable carries data, and the power-on sequence. **Do not run the flash command until you see the APX device.**

B6 Flash

Using **Tanna's board-config name** — `<BOARD>` is theirs, not a guess:

```
cd ~/nvidia/Linux_for_Tegra
sudo ./flash.sh <BOARD> mmcblk0p1
```

Takes 10–20 minutes. Success looks like: `*** The target ... has been flashed successfully. ***`

> **Do not interrupt power or unplug USB during this**
> This is the only genuinely risky window in the whole procedure.

B7 First boot

```
cat /etc/nv_tegra_release
ls /dev/video*                # is the camera present?
```

Power off, remove the USB cable, connect HDMI + keyboard + power. Complete the Ubuntu setup wizard if it appears.

B8 Install JetPack components

The flash writes the OS only. CUDA, cuDNN and TensorRT come afterwards, over the network:

```
sudo apt update
sudo apt install -y nvidia-jetpack        # ~14 GB — see warning
```

> **This will not fit on 16 GB eMMC**
> Either install selectively — `cuda-toolkit-10-2`, `libcudnn8`, `tensorrt`, skipping samples, docs and VisionWorks — or complete the A7 SD relocation **first** and point `/var/cache/apt` and the bulky directories at the card.

B9 Post-flash configuration

```
sudo systemctl set-default multi-user.target      # frees ~1 GB of 4 GB
# 8 GB swap on the SD — see step A7
sudo nvpmodel -m 0 && sudo jetson_clocks          # MAXN + pinned clocks
sudo apt install -y espeak libespeak1 alsa-utils

python3 -m venv --system-site-packages ~/echopath-env
source ~/echopath-env/bin/activate
export PATH=/usr/local/cuda/bin:$PATH
export CUDA_ROOT=/usr/local/cuda
pip install pycuda==2021.1 pyttsx3==2.90
```

`--system-site-packages` is essential: it lets the venv see JetPack's `tensorrt` and `cv2` while giving you somewhere to put `pycuda`.

B10 Verify before trusting it

- ☐ `free -h && df -h /` — >40 GB free, 8 GB swap active
- ☐ `ls /dev/video*` and the `nvarguscamerasrc` pipeline from A5
- ☐ `aplay -l` shows the USB dongle; a test WAV is audible in the earphones
- ☐ `python3 -c "import tensorrt, cv2; print(tensorrt.__version__, cv2.__version__)"`
- ☐ `python3 -c "import cv2; print('GStreamer' in cv2.getBuildInformation())"` → **True**
- ☐ `python3 -c "import pycuda.autoinit; print('pycuda ok')"`
- ☐ Fan spins under `jetson_clocks`

B11 Image the card immediately

Once B10 passes, back the card up to the 64 GB pendrive:

```
# on the Linux host, card in a reader
sudo dd if=/dev/sdX of=/media/pendrive/echopath_working.img bs=4M status=progress
```

A corrupted card then becomes a 20-minute restore instead of repeating Part B.

Troubleshooting

Symptom	Likely cause and fix
No APX device in `lsusb`	Not in recovery mode, or a charge-only USB cable. Try a different cable first — this is the commonest cause.
Flash fails partway	Bad cable, a USB hub (use a direct port on the host), or a VM. Repeat B5–B6.
Flashes but will not boot	**Wrong board config.** Go back to Tanna — do not try other names at random.
Boots but no camera	Stock L4T device tree instead of the vendor BSP.
Random reboots	Under-powered supply. Needs a genuine 5 V / 4 A.
TensorRT build dies silently	Out of memory — the OOM killer. Check `dmesg | grep -i "killed process"`. Fix: headless boot, 8 GB swap, `--workspace=512`, one engine at a time.
CSI pipeline stops working later	Someone ran `pip install opencv-python`. It shadows JetPack's GStreamer build. Uninstall it.

> **Bricking is unlikely**
> Recovery mode is almost always re-enterable, so a failed flash is usually fixed by repeating B5–B6. It is hard to permanently damage a Jetson this way.

Never do these

- `pip install opencv-python` **on the device** — silently kills the CSI camera path
- **Guess the board-config name** — the one action that can leave the board non-booting
- **Flash from a VM** — USB re-enumeration mid-flash
- **Put swap on the eMMC** — no space, and it wears the chip
- **Build both TensorRT engines at once** — doubles peak memory, guaranteed OOM
- **Use** `--int8` — Maxwell has no DP4A; it warns and silently falls back to FP16

Do today, before sourcing anything

1. **Run Part A.** Thirty minutes, and it may make Part B unnecessary.
2. **Email Tanna** the five questions in B1 — especially whether they will flash it for you.

EchoPath — Jetson bring-up runbook · prepared 14 August 2026 · unit serial 1010425035310. Vendor-specific items (board-config name, recovery jumper location, BSP layout) must come from Tanna TechBiz and are marked as such. Everything else is standard L4T R32.7 practice for the Jetson Nano production module.